SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-015: Student Records Export to CSV

Field	Details
Change Request ID	CR-015
Change Request Title	Student Records Export to CSV
Student Name	T.Manoja
Student ID	JF/ICT/24/12
Implementation Status	Implemented in frontend code; execution/testing status to be confirmed
Date	15 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System requires a facility that allows authorized administrators to export available student records into a portable CSV file. This supports administrative reporting, record keeping, and reviewing student information outside the system.

The exported file is expected to contain Student ID, Registration Number, First Name, Last Name, Email, and Phone Number. It must include column headers, use a meaningful filename such as students.csv, and handle multiple records and an empty student list safely.

1.2 Existing Behaviour

- Administrators can view student records in the existing Admin Dashboard.
- The dashboard loads student data through the existing GET /api/students/ request.
- The dashboard already supports client-side student search and existing Add, Edit, and Delete operations.
- A dedicated CSV export control was not present in the supplied DashboardPage.jsx before this change.
- Existing student CRUD operations use student_id to identify records.

1.3 Requested Change

- Add an Export CSV button to the administrator's Students heading area.
- Generate a CSV file from the student records already loaded in the dashboard.
- Include the six required columns and a header row.
- Download the file as students.csv.
- Handle empty student data without a runtime error.
- Preserve search, viewing, Add, Edit, Delete, authentication, and authorization behaviour.

2. Implementation

2.1 Implementation Summary

The supplied DashboardPage.jsx already loads student records into the students state through apiRequest("/api/students/", { method: "GET" }). The CSV export enhancement can reuse this data, so a separate export endpoint is not required if the existing endpoint returns all required fields and enforces the existing administrator authorization.

A handleExportCSV function maps each student record to the requested CSV columns, escapes quotes and delimiters, creates a UTF-8 CSV Blob, and initiates a browser download named students.csv. A UTF-8 BOM is included to improve compatibility with spreadsheet applications. The button is disabled while records are loading or when the student-loading request has an error.

The new button is intended to appear next to the existing + Add Student button. The implementation should be tested in the actual development environment before the status is changed to fully completed.

2.2 Affected Files

File	Change	Purpose
frontend/src/pages/DashboardPage.jsx	Modified	Adds handleExportCSV and an Export CSV button in the Students heading area; reuses the existing students state.
frontend/src/styles/dashboard.css	Modified	Adds styling and responsive layout rules for the export button and heading action group.

2.3 Technical Changes

2.3.1 CSV Data Mapping

The export maps each loaded student to these fields:

- student.student_id → Student ID
- student.registration_no → Registration Number
- student.first_name → First Name
- student.last_name → Last Name
- student.email → Email
- student.phone_number → Phone Number

2.3.2 CSV Formatting and Download

- The first CSV row contains the six column headers.
- Each student is written as one CSV row.
- Values are quoted and embedded double quotes are escaped by doubling them.
- A UTF-8 BOM is included for spreadsheet compatibility.
- The browser downloads the generated Blob as students.csv.
- An empty students array produces a header-only CSV instead of throwing an error.

2.3.3 Frontend Placement

In DashboardPage.jsx, add handleExportCSV inside the DashboardPage component after handleClearSearch and before the // Form handling section. Replace only the existing Add Student button block inside with the action group containing Export CSV and + Add Student. Keep the surrounding heading, description, student table, search, modal, and CRUD handlers.

In dashboard.css, add the .students-heading-actions and .export-csv-button styles at the end of the file, including responsive rules for narrower screens.

2.3.4 Security and Authorization

The CSV button is a frontend control and does not itself enforce security. Student records are obtained through the existing GET /api/students/ endpoint. The backend must continue to require authentication and administrator authorization. The export must not use a public or unprotected endpoint. No authentication or authorization rules should be weakened by this change.

The CSV escaping also reduces the risk of spreadsheet formula interpretation for values beginning with formula characters. This is a client-side export safeguard, not a replacement for backend access control.

2.4 Implementation Logic

1. The dashboard retrieves student records through the existing protected student API.
2. The administrator clicks Export CSV.
3. The handler reads the already-loaded students array.
4. The handler constructs the required header row and maps each student to six fields.
5. CSV values are escaped to preserve commas, quotes, and line breaks.
6. The CSV text is placed in a UTF-8 Blob.
7. A temporary download link triggers the students.csv download.
8. The export does not send update or delete requests and does not modify the original records.

2.5 Impact Analysis

Area	Impact
Frontend	Low to moderate: adds a button, CSV generation logic, and responsive styling.
Backend/API	No change expected if the existing student-list endpoint supplies all required fields and enforces authorization.
Database	No schema change or migration required.
Authentication/Authorization	Existing controls must remain in place; no change to login or role rules.
Student CRUD	No intended change; Add, Edit, Delete and search must be regression-tested.
Performance	Low for normal student lists; the CSV is generated in the browser from loaded data.
Deployment	Frontend redeployment only if no backend changes are required.

3. Testing & Verification

3.1 Testing Approach

Testing should be performed in the development environment using an authorized administrator account. The tests below are proposed verification cases. Their actual results must be filled in after executing them; they are not represented as already passed.

Environment: Development. Frontend: React. Backend: Node.js/Express. Database: MySQL. Browser: Chrome or Microsoft Edge.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result
TC-01	Open Admin Dashboard as an authorized administrator.	Export CSV button is visible.	To be tested
TC-02	Click Export CSV with student records loaded.	A file named students.csv downloads.	To be tested
TC-03	Open the exported CSV.	The six required column headers appear in the correct order.	To be tested
TC-04	Export several student records.	Each record appears in a separate row with correct values.	To be tested
TC-05	Export when the student list is empty.	A header-only CSV downloads without a runtime error.	To be tested
TC-06	Export values containing commas, quotes, or line breaks.	CSV quoting preserves the field contents.	To be tested
TC-07	Try accessing the student-list API without authentication or with an unauthorized role.	The backend rejects the request according to existing security rules.	To be tested
TC-08	Use search, Add, Edit, and Delete after adding export.	Existing functionality continues to work.	To be tested
TC-09	Compare exported values with records returned by the API.	Exported data matches the available loaded records.	To be tested
TC-10	Export records and then check the database.	Export has not modified or deleted original records.	To be tested

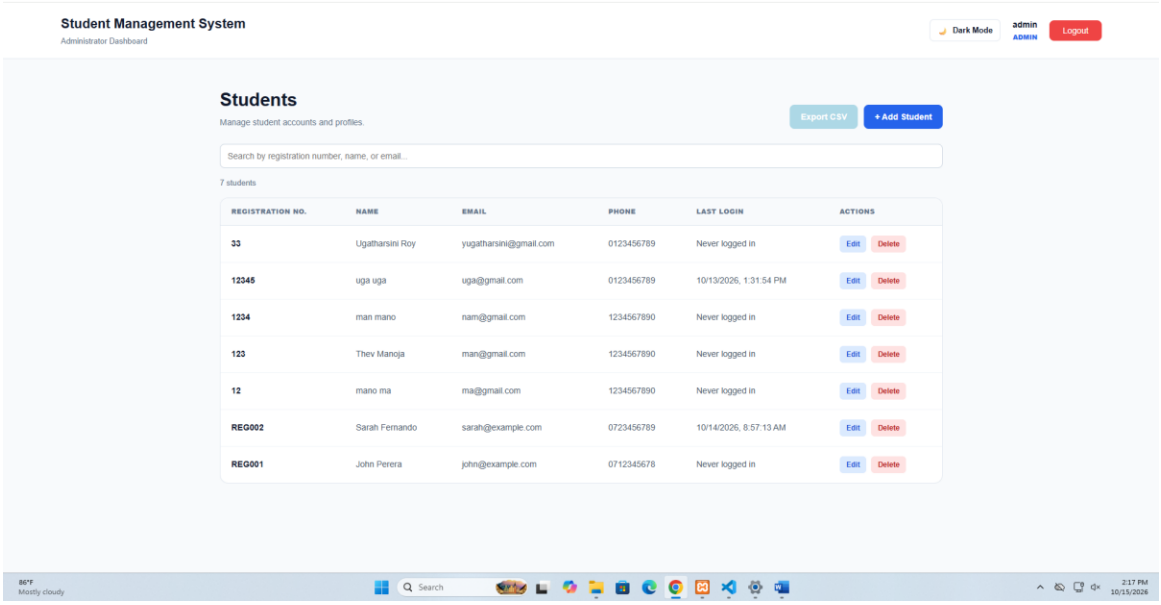
4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method
AC-001	Authorized administrator can see Export CSV.	Dashboard UI inspection
AC-002	CSV downloads successfully with student records.	Browser download test
AC-003	CSV contains the required headers.	Open CSV and inspect header row
AC-004	CSV values match Student ID, Registration Number, First Name, Last Name, Email, and Phone Number.	Compare CSV with API records
AC-005	Multiple records export correctly.	Multi-record test
AC-006	File uses .csv extension and meaningful filename.	Check downloaded filename
AC-007	Export matches the available student records.	Compare records
AC-008	Empty list is handled without a runtime error.	Empty-list test
AC-009	Unauthorized users cannot retrieve student records.	Backend authorization test
AC-010	Existing CRUD functionality continues to work.	Regression test

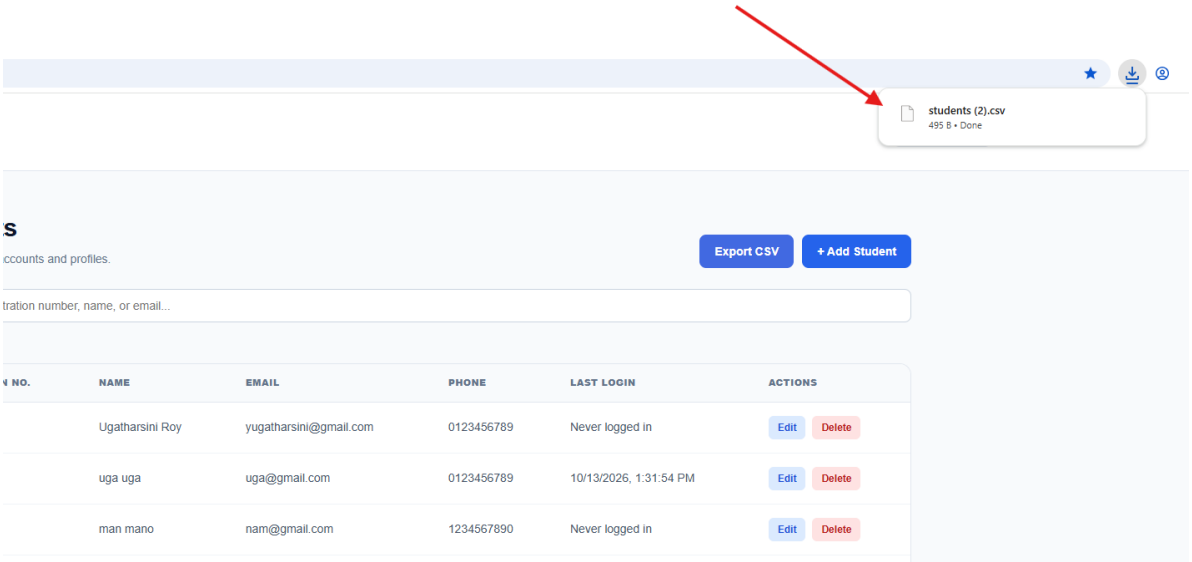
AC-011	Existing authentication and authorization remain operational.	Authentication/authorization regression test
AC-012	Export does not modify or delete original records.	Database comparison before/after export

5. Evidence

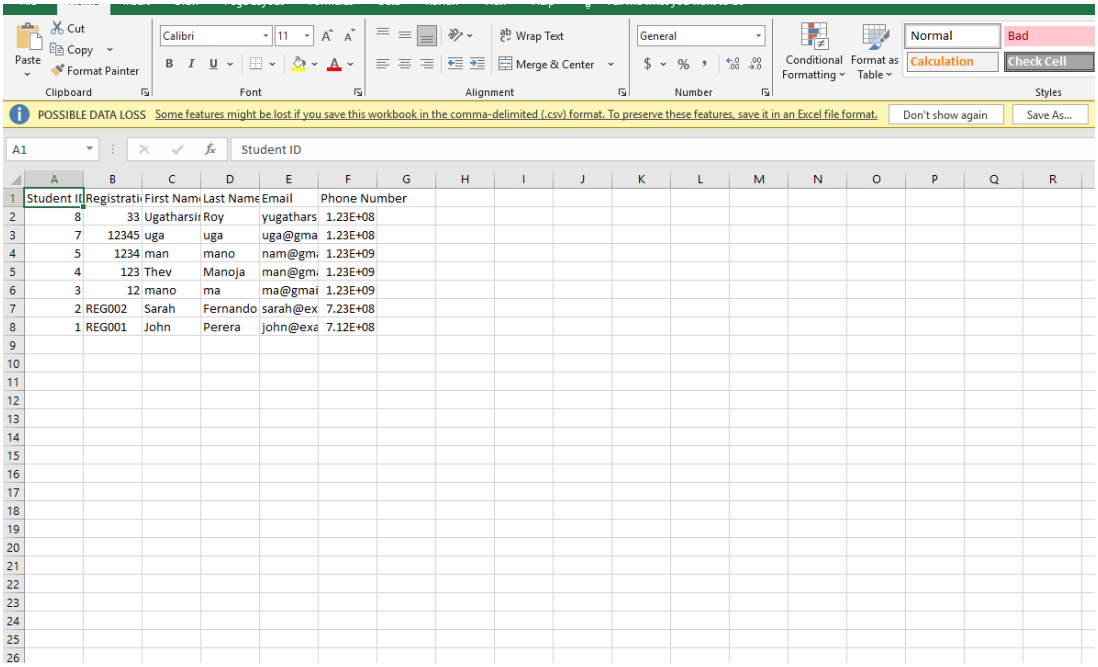
Evidence 01 – Export CSV Button



Evidence 02 – Successful Download

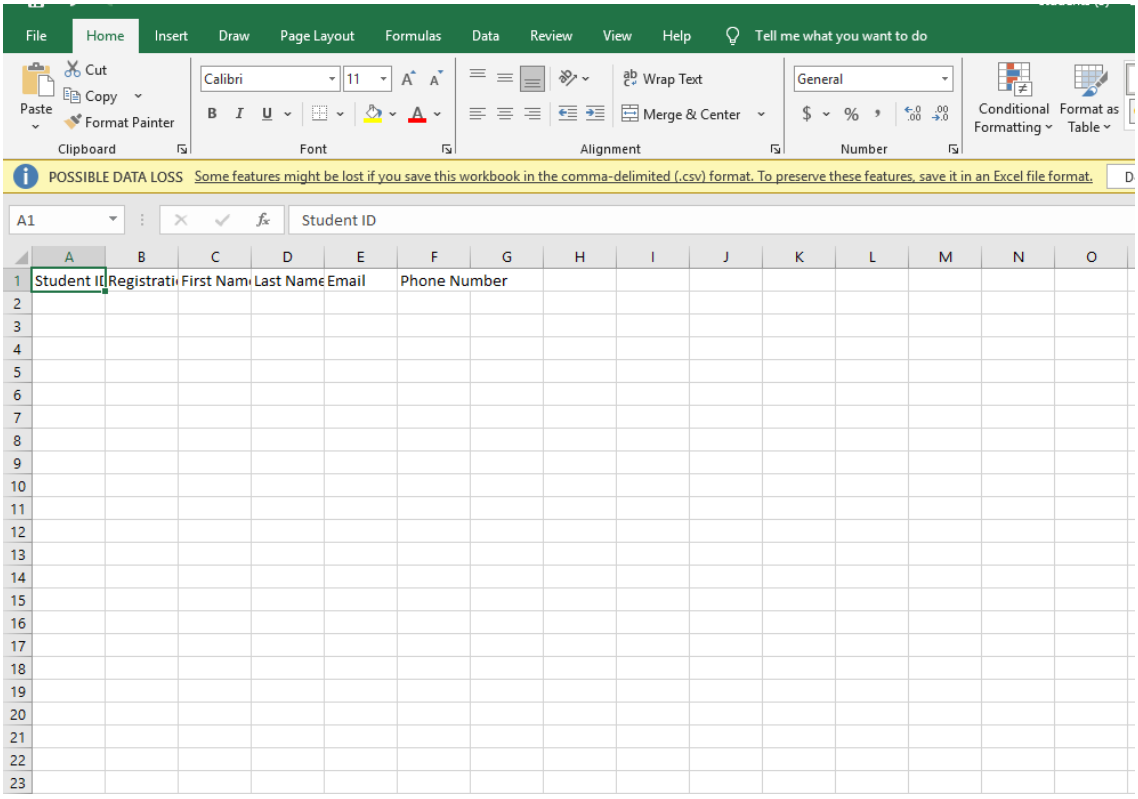


Evidence 03 – CSV Contents



Student ID	Registrati	First Nam	Last Name	Email	Phone Number
8	33	Ugatharsi	Roy	yugathars	1.23E+08
7	12345	uga	uga	uga@gma	1.23E+08
5	1234	man	mano	nam@gm	1.23E+09
4	123	Thev	Manoja	man@gm	1.23E+09
3	12	mano	ma	ma@gmai	1.23E+09
2	REG002	Sarah	Fernando	sarah@ex	7.23E+08
1	REG001	John	Perera	john@exa	7.12E+08

Evidence 04 – Empty Student List



Student ID	Registrati	First Nam	Last Name	Email	Phone Number

Evidence 05 – Authorization Test

Capture the protected API rejection for an unauthenticated or unauthorized request, without exposing tokens or sensitive student data. Related criteria: AC-009 and AC-011.

6. Final Result

Item	Result
Requirements	Implementation approach documented; verify in running application
Implementation	Frontend changes identified for DashboardPage.jsx and dashboard.css
Testing	Pending actual execution
Acceptance Criteria	Pending verification
Existing Functionality	Must be confirmed by regression testing
Final Status	Pending testing and evidence

6.1 Final Summary

CR-015 adds a CSV export facility to the administrator's student management interface. The implementation uses the student records already loaded in DashboardPage.jsx, generates a CSV with the six required columns, and downloads the file as students.csv. It is designed to handle multiple records and an empty student list while leaving the existing student CRUD operations and database records unchanged.

The expected implementation changes are limited to frontend/src/pages/DashboardPage.jsx and frontend/src/styles/dashboard.css, provided the existing GET /api/students/ endpoint returns the required fields and enforces administrator authorization. No database migration is expected. Final completion should be recorded only after the export, authorization behaviour, and regression tests have been executed.

Student Name: T.Manoja

Student ID: JF/ICT/24/12

Date: 15 October 2026

Signature: _____